Agentic架构 2：工具使用（Tool Use）

> **工具使用(Tool Use)** —— 将大语言模型的**推理能力与现实、动态世界连接**起来的桥梁。没有工具，LLM就是一个封闭的系统，受限于其训练数据中冻结的知识。它无法知道今天的天气、当前的股价，或您公司数据库中订单的状态。通过赋予Agent**使用工具**的能力，我们使其能够克服这一根本限制，使其能够**查询API、搜索数据库并访问实时信息**，从而提供不仅经过推理，而且**事实准确、及时且相关**的答案。

定义

工具使用架构为基于LLM的Agent配备了**调用外部函数或API（即"工具"）的能力**。Agent自主判断何时仅凭其**内部知识**无法回答用户的查询，并决定调用哪个**合适的工具**来查找必要信息。

高层工作流

1. **接收查询：** Agent接收来自用户的**请求**。
2. **决策：** Agent**分析查询**及其可用的工具。它判断是否需要使用工具才能准确回答问题。
3. **执行：** 如果需要工具，Agent会**格式化**对该工具的调用（例如，一个带有正确参数的特定函数）。
4. **观察：** 系统**执行工具调用**，并将**结果**（即"观察结果"）返回给Agent。
5. **综合：** Agent将工具的**输出整合**到其推理过程中，为用户生成最终的、有依据的答案。

适用场景 / 应用

- **研究助手：** 通过**使用网络搜索API**，回答需要最新信息的问题。
- **企业助手：** **查询内部公司数据库**，回答诸如"上周有多少新用户注册？"之类的问题。
- **科学与数学任务：** 使用计算器或WolframAlpha等**计算引擎**，进行LLM通常难以处理的精确计算。

优势与劣势

- **优势：**
  - **事实依据：** 通过获取**真实、实时**的数据，大幅减少幻觉。
  - **可扩展性：** 只需添加新工具，即**可持续扩展**Agent的能力。
- **劣势：**
  - **集成开销：** 需要仔细的"**管道(Pipeline)**"工作来定义工具、处理API密钥以及管理潜在的工具故障。
  - **工具信任：** Agent答案的质量取决于其**所用工具的可靠性和准确性**。Agent必须信任其工具提供的信息是正确的。

阶段 0：基础与环境搭建

首先进行环境搭建。包括安装必要的库，以及为 Nebius、LangSmith 和我们将要使用的特定工具配置 API 密钥。

步骤 0.1：安装核心库

- 用于**编排**的标准库集（`langchain-nebius`、`langgraph`）、**环境管理库**（`python-dotenv`）和**打印美化库**（`rich`）

- 安装 `tavily-python`，它提供了一个易于使用的 API 接口，用于**强大的网络搜索工具**，把这个工具提供给我们的 Agent。

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
tavily-python
```

步骤 0.2：导入库与配置密钥

导入必要的模块，并使用 `python-dotenv` 加载我们的 API 密钥，需要 **Nebius（用于 LLM）、LangSmith（用于追踪）和 Tavily（用于网络搜索工具）的密钥。**

需要执行的操作： 在此目录下创建一个 `.env` 文件，并填入密钥：

```
NEBIUS_API_KEY="your_nebius_api_key_here"
LANGCHAIN_API_KEY="your_langsmith_api_key_here"
TAVILY_API_KEY="your_tavily_api_key_here"
```

```
import os
import json
from typing import List, Annotated, TypedDict, Optional
from dotenv import load_dotenv

# LangChain 组件
from langchain_nebius import ChatNebius
from langchain_community.tools.tavily_search import TavilySearchResults
from langchain_core.messages import BaseMessage, ToolMessage
from pydantic import BaseModel, Field

# LangGraph 组件
from langgraph.graph import StateGraph, END
from langgraph.graph.message import AnyMessage, add_messages
from langgraph.prebuilt import ToolNode

# 美化打印输出
from rich.console import Console
from rich.markdown import Markdown

# --- API 密钥与追踪配置 ---
load_dotenv()  # 加载 .env 文件中的环境变量

# 配置 LangSmith 追踪功能
os.environ["LANGCHAIN_TRACING_V2"] = "true"  # 启用 LangSmith 追踪 v2 版本
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - Tool Use
(Nebius)"  # 设置追踪项目名称

# 校验必需的环境变量是否已配置
for key in ["NEBIUS_API_KEY", "LANGCHAIN_API_KEY", "TAVILY_API_KEY"]:
    if not os.environ.get(key):
```

```
        print(f"{key} not found. Please create a .env file and set it.")  # 缺失密钥提示

print("Environment variables loaded and tracing is set up.")
```

阶段 1：定义 Agent 的工具箱

Agent 的能力取决于它可以访问的工具。本阶段定义并测试提供给 Agent 的特定工具：**实时网络搜索**。

步骤 1.1：创建并测试网络搜索工具

实例化 `TavilySearchResults` 工具。定义工具**最关键**的部分是其**描述（description）**。LLM 使用这个**自然语言描述**来**理解工具的功能以及何时应该使用它**。清晰、准确的描述对于 Agent 做出正确的决策至关重要。然后**直接测试**该工具，看看其原始输出的样子。

```
# 初始化搜索工具，设置最大返回结果数以保持上下文简洁
search_tool = TavilySearchResults(max_results=2)

# 为工具指定清晰的名称和描述，这对智能体正确理解和使用工具至关重要
search_tool.name = "web_search"
search_tool.description = "A tool that can be used to search the internet for up-to-date information on any topic, including news, events, and current affairs."

# 将工具放入工具列表，便于后续传递给智能体
tools = [search_tool]
print(f"Tool '{search_tool.name}' created with description: '{search_tool.description}'")  # 提示工具创建完成

# 初始化 Rich 控制台，用于美化打印输出
console = Console()

# 直接测试工具，以了解其输出格式
print("\n--- Testing the tool directly ---")

# 定义测试查询内容
test_query = "What was the score of the last Super Bowl?"

# 调用搜索工具执行查询
test_result = search_tool.invoke({"query": test_query})

# 使用 Rich 控制台以绿色粗体显示查询内容和结果
console.print(f"[bold green]Query:[/bold green] {test_query}")  # 输出测试查询
console.print("\n[bold green]Result:[/bold green]")  # 输出结果标题
console.print(test_result)  # 输出搜索结果
```

> **关于输出的讨论：** 测试显示了 `web_search` 工具的原始输出。它返回一个**字典列表**，其中每个字典包含搜索结果的 URL 和内容片段。这种**结构化的信息**正是 Agent 决定使用该工具后，将作为"**观察结**

> **果**"接收到的内容。有一个**可用的工具**，就可以构建将学习如何使用它的 Agent。

阶段 2：使用 LangGraph 构建工具使用型 Agent

构建 Agent 工作流。这涉及让 LLM 了解工具的存在，并创建一个使其能够**循环执行"思考-执行-观察"周期**的图结构，这正是工具使用的**核心本质**。

步骤 2.1：定义图状态

工具使用型 Agent 的状态通常是一个**消息列表**，代表**对话历史**。这个历史记录包括**用户的问题、Agent 的思考和工具调用，以及这些工具返回的结果**。使用一个 `TypedDict` 来容纳任意类型的 LangChain 消息。

```
class AgentState(TypedDict):
    """
    智能体状态结构，用于管理对话历史。

    使用 Annotated 类型与 add_messages 函数配合，实现消息列表的自动合并更新。
    当向状态中添加新消息时，LangGraph 会自动将新消息追加到现有消息列表末尾。
    """
    messages: Annotated[list[AnyMessage], add_messages]  # 对话消息列表，支持
自动追加更新

print("AgentState TypedDict defined to manage conversation history.")
```

步骤 2.2：将工具绑定到 LLM

这是**让 LLM"感知"到工具**的关键步骤。使用 `.bind_tools()` 方法，将工具的**名称和描述**传递给 LLM 的**系统提示词**。这使得模型能够根据工具描述，在其**内部逻辑**中决定**何时调用工具**。

```
# 初始化 Nebius 大语言模型，使用 Llama 3.1 8B 指令微调版本
# 设置 temperature=0 使输出结果更具确定性和一致性，适合工具调用场景
llm = ChatNebius(model="meta-llama/Meta-Llama-3.1-8B-Instruct",
temperature=0)

# 将工具绑定到 LLM，使模型具备工具感知能力
# 绑定后 LLM 可以根据用户输入自主决定是否需要调用工具以及调用哪个工具
llm_with_tools = llm.bind_tools(tools)

print("LLM has been bound with the provided tools.")
```

步骤 2.3：定义 Agent 节点

图结构将有两个主要节点：

1. `agent_node`： 这是 **"大脑"**。它使用当前的**对话历史**调用 LLM。LLM 的响应要么是**最终答案**，要么是**调用工具的请求**。

2. **`tool_node`**： 这是"**双手**"。它接收来自 `agent_node` 的**工具调用请求**，**执行相应的工具**，并返回输出。我们将使用 LangGraph 预构建的 **`ToolNode`** 来实现此功能。

```
def agent_node(state: AgentState):
    """
    智能体节点，负责调用 LLM 来决定下一步行动。

    该节点将当前对话历史传递给 LLM，LLM 会根据用户输入和可用工具，自主决定是直接回答还是调用某个工具来获取信息。
    """
    console.print("--- AGENT: Thinking... ---")
    # 调用已绑定工具的 LLM，传入当前消息列表，获取模型的响应
    response = llm_with_tools.invoke(state["messages"])
    # 返回响应消息，LangGraph 会自动将其追加到状态的消息列表中
    return {"messages": [response]}

# ToolNode 是 LangGraph 预构建的节点，负责执行工具调用
# 当 LLM 决定调用工具时，ToolNode 会自动执行对应的工具并返回结果
tool_node = ToolNode(tools)

print("Agent node and Tool node have been defined.")
```

步骤 2.4：定义条件路由

在 `agent_node` 执行之后，需要**决定下一步去向**。**路由函数**检查来自 Agent 的最后一条消息。如果该消息包含 **`tool_calls` 属性**，则表示 Agent 想要使用工具，因此我们路由到 `tool_node`。如果没有，则表示 Agent 已有最终答案，我们可以结束工作流。

```
def router_function(state: AgentState) -> str:
    """
    路由函数，根据智能体的最后一条消息决定下一步流程走向。

    该函数检查智能体最新响应中是否包含工具调用请求：
    - 如果有工具调用，则路由到工具执行节点
    - 如果没有工具调用，说明智能体已给出最终答案，结束流程
    """
    # 获取状态中最新的一条消息（即智能体的最近一次响应）
    last_message = state["messages"][-1]

    # 检查消息中是否包含工具调用请求
    if last_message.tool_calls:
        # 智能体请求调用工具，路由到工具节点
        console.print("--- ROUTER: Decision is to call a tool. ---")
        return "call_tool"  # 返回工具节点标识
    else:
        # 智能体已给出最终答案，无工具调用请求，结束流程
        console.print("--- ROUTER: Decision is to finish. ---")
        return "__end__"  # 返回结束标识，LangGraph 内置常量
```

```
print("Router function defined.")  # 提示路由函数已定义完成
```

阶段 3：组装并运行工作流

把所有组件连接成一个**完整的、可执行的图结构**，并在一个迫使 Agent 使用其新网络搜索能力的查询上运行它。

步骤 3.1：构建并可视化图结构

创建 `StateGraph`，并添加节点和边。关键部分是**条件边**，它使用我们的 `router_function` 创建 Agent 的主要推理循环：`agent → router → tool → agent`。

```
# 创建状态图构建器，指定使用 AgentState 作为状态结构
graph_builder = StateGraph(AgentState)

# 向图中添加节点
graph_builder.add_node("agent", agent_node)      # 智能体节点：负责调用 LLM 决定行动
graph_builder.add_node("call_tool", tool_node)   # 工具执行节点：负责执行 LLM 请求的工具调用

# 设置入口节点为智能体节点
graph_builder.set_entry_point("agent")

# 添加条件边：从智能体节点出发，根据路由函数的返回值决定下一跳
graph_builder.add_conditional_edges(
    "agent",
    router_function,  # 路由函数，返回 "call_tool" 或 "__end__"
)

# 添加从工具节点返回智能体节点的边，形成循环
# 这使得智能体可以在执行工具后继续处理结果，支持多轮工具调用
graph_builder.add_edge("call_tool", "agent")

# 编译图，生成可执行的应用实例
tool_agent_app = graph_builder.compile()

print("Tool-using agent graph compiled successfully!")

# 可视化图结构（用于调试和展示）
try:
    from IPython.display import Image, display  # 导入 IPython 可视化相关模块
    png_image = tool_agent_app.get_graph().draw_png()  # 获取图的 PNG 图像数据
    display(Image(png_image))  # 在 Jupyter 环境中显示图像
except Exception as e:
    # 如果可视化失败（如缺少 pygraphviz 依赖），打印错误信息，不影响主流程运行
```

```
    print(f"Graph visualization failed: {e}. Please ensure pygraphviz is
installed.")
```

> **关于输出的讨论：** 编译后的图结构已准备就绪。可视化清晰地展示了 **Agent 的推理循环**。流程从 `agent` 节点开始。然后**条件边**（用菱形表示）决定路由方向。如果需要**工具**，则转到 `call_tool`，并将输出反馈回 `agent` 进行综合。如果不需要工具，则流程转到 `__end__`。这种结构完美地实现了**工具使用**模式。

步骤 3.2：端到端执行

用一个 Agent 从训练数据中无法获知的问题来运行它，迫使其使用**网络搜索工具**。流式传输中间步骤，以观察其推理过程逐步展开。

```
# 定义用户查询内容：询问苹果最新 WWDC 活动的主要公告
user_query = "What were the main announcements from Apple's latest WWDC
event?"

# 构建初始状态输入，使用元组格式 ("user", 消息内容) 来标识消息来源角色
initial_input = {"messages": [("user", user_query)]}

# 在控制台以青色粗体显示流程启动信息
console.print(f"[bold cyan]🚀 Kicking off Tool Use workflow for request:
[/bold cyan] '{user_query}'\n")

# 流式执行工具智能体工作流，以 "values" 模式逐个接收状态快照
for chunk in tool_agent_app.stream(initial_input, stream_mode="values"):
    # 获取当前状态快照中的最后一条消息（即最新响应），使用 pretty_print() 方法以美观格
式打印消息内容
    chunk["messages"][-1].pretty_print()
    # 在每条消息后打印分隔线，增强可读性
    console.print("\n---\n")

console.print("\n[bold green]✅ Tool Use workflow complete![/bold green]")
```

阶段 4：评估

现在 Agent 已经运行完毕，可以评估其性能了。对于工具使用型 Agent，关心两点：**它是否正确使用了工具，以及从工具输出综合而来的最终答案是否高质量。**

步骤 4.1：分析执行轨迹

通过查看上一步的流式输出，**追溯 Agent 的确切思考过程**。输出显示了流经**图状态**的不同消息类型（**带有 `tool_calls` 的 `AIMessage`、带有结果的 `ToolMessage`**）。

> **关于输出的讨论：** 执行轨迹清晰地展示了工具使用模式的实际运作：
>
> 1. 打印的第一条消息来自 **`agent` 节点**。它是一个包含 `tool_calls` 属性的 `AIMessage`，表明 LLM 正确决定使用 `web_search` 工具。

> 2. 下一条消息是 `ToolMessage`。这是 **`tool_node` 在执行搜索并返回原始结果**后的输出。
> 3. 最后一条消息是另一个 `AIMessage`，但这次没有 `tool_calls`。这是 Agent 将 `ToolMessage` 中的信息综合成一个连贯的最终答案呈现给用户。
>
> 该轨迹证实了 Agent 的**逻辑和图结构的路由**都完美运行。

步骤 4.2：使用 LLM 作为评判者进行评估

创建一个"评判者" LLM，为 Agent 的性能提供结构化的**定量评估**。评估标准将专门针对工具使用的质量进行定制。

```
class ToolUseEvaluation(BaseModel):
    """用于评估智能体工具使用能力与最终回答质量的结构。"""
    tool_selection_score: int = Field(description="工具选择正确性评分，1-5分（是否正确选择了适合任务的工具）。")
    tool_input_score: int = Field(description="工具输入质量评分，1-5分（输入格式是否规范、信息是否充分相关）。")
    synthesis_quality_score: int = Field(description="综合整合质量评分，1-5分（智能体将工具输出整合到最终回答中的效果）。")
    justification: str = Field(description="各项评分的简要说明。")

# 将基础 LLM 包装为结构化输出模型，确保输出符合 ToolUseEvaluation 的结构定义
judge_llm = llm.with_structured_output(ToolUseEvaluation)

# 为了进行评估，需要重构完整的对话轨迹，调用已编译的智能体应用，传入初始输入，获取完整执行结果
final_answer = tool_agent_app.invoke(initial_input)

# 将对话历史中的每条消息转换为字符串格式，用于评估，提取每条消息的类型、内容以及工具调用信息（如果存在）
conversation_trace = "\n".join([
    f"{m.type}: {m.content or ''} {getattr(m, 'tool_calls', '')}"
    for m in final_answer['messages']
])

def evaluate_tool_use(trace: str):
    """对智能体的工具使用能力进行评估，返回各项评分和说明。"""
    # 构建评估提示词，要求模型扮演 AI 智能体评审专家角色
    prompt = f"""You are an expert judge of AI agents. Evaluate the following conversation trace based on the agent's tool use on a scale of 1-5. Provide a brief justification.

    Conversation Trace:
```

> **关于输出的讨论： LLM 作为评判者为 Agent 的性能提供了结构化的、有理有据的评估**。在三个类别——`tool_selection_score`（工具选择评分）、`tool_input_score`（工具输入评分）和 `synthesis_quality_score`（综合质量评分）——中获得的高分证实了我们的 Agent 不仅使用了工具，而且**有效地使用了工具**。它正确识别了网络搜索的需求，制定了相关的查询，并成功地将检索到的事实综合成一个**有用且准确的最终答案**。这种自动化评估使我们对实现的健壮性充满信心。

结论

在本笔记中，我们基于**工具使用**架构构建了一个完整且可运行的 Agent。我们成功地为基于 Nebius 的 LLM 配备了一个**网络搜索工具**，并使用 **LangGraph** 创建了一个健壮的推理循环，使 Agent 能够决定何时以及如何使用该工具。

端到端的执行以及后续的评估证明了这种模式的巨大价值。通过将我们的 Agent **连接到实时的外部信息**，我们从根本上克服了**静态训练数据**的局限性。Agent 不再仅仅是一个**推理者**；它更是一个**研究员**，能够提供有据可依、事实准确且实时的答案。这种架构是构建几乎任何实用的、现实世界 AI 助手的基础构建块。